

Avaliação de Modelos de Linguagem Orientados por Ações em uma Pipeline de QA para Aplicações Web

Henrique Oliveira da Cunha Franco¹, Humberto Torres Marques Neto¹

¹Instituto de Informática – Pontifícia Universidade Católica de Minas Gerais
(PUC Minas)

henrique.franco.1448652@sga.pucminas.br, humberto@pucminas.br

Abstract. *This work presents E2P, a local pipeline for web project understanding, interface exploration, test planning, Playwright generation, and defect assessment. A quantized local language model selects bounded actions, while recorded evidence supports human review. Three application runs produced six tests, five passing, but exposed weak assertions and a false defect hypothesis. A later Form Validator retest admitted two flows after correcting the admission filter. Results support operational feasibility, not autonomous QA reliability: reproducing an observation does not establish that it violates an expected behavior.*

Resumo. *Este trabalho apresenta o E2P, uma pipeline local para entendimento de projetos web, exploração da interface, planejamento, geração Playwright e análise de defeitos. Um modelo de linguagem local quantizado seleciona ações limitadas, com evidências para revisão humana. Três execuções produziram seis testes, cinco aprovados, mas revelaram asserções fracas e uma hipótese falsa de defeito. Um reteste no Form Validator admitiu dois fluxos após corrigir o filtro. Os resultados sustentam viabilidade operacional, não confiabilidade autônoma: reproduzir uma observação não comprova violação do comportamento esperado.*

1. Introdução

Testes ponta a ponta (*End-to-End*, E2E) exercitam jornadas de uso sobre um sistema integrado. Sua utilidade depende não apenas de executar interações, mas também de selecionar comportamentos relevantes e verificar seus resultados. Para um projeto web simples ou moderado sem uma rotina de garantia de qualidade (*Quality Assurance*, QA), preparar essa automação envolve entender o produto, configurar seu ambiente, explorar suas funções, especificar casos e interpretar falhas. Automatizar somente a escrita de scripts deixa parte importante desse trabalho em aberto.

Modelos de linguagem de grande porte (*Large Language Models*, LLMs) têm sido investigados em geração de testes, reprodução de defeitos e apoio à manutenção de software [Wang et al. 2024, Kang et al. 2023]. Entretanto, uma resposta textual plausível não comprova que controles existem, que uma jornada foi executada ou que uma expectativa é correta. Estudos de agentes web evidenciam dificuldades na conclusão de tarefas realistas [Zhou et al. 2024, Drouin et al. 2024]. Em QA, essas dificuldades se somam ao risco de transformar um comportamento normal em relato de defeito.

Neste trabalho, *orientado por ações* caracteriza o papel do modelo na orquestração: a partir da observação da aplicação, ele escolhe uma operação em um catálogo explícito, executada e verificada por ferramentas. Não se trata de uma nova arquitetura neural ou de um modelo treinado neste estudo. O protótipo Action E2E Prototype (E2P) combina documentação e trechos de código, estado estruturado da interface, percepção visual opcional, ações limitadas e revisão humana. Seu propósito é apoiar a implantação de uma pipeline de QA em aplicações web de pequeno a médio porte, e não substituir integralmente o julgamento de um profissional.

O problema de pesquisa é: *em que medida uma pipeline local baseada em modelos orientados por ações consegue transformar a exploração de uma aplicação web em casos executáveis e hipóteses de defeito sustentadas por evidência?* O objetivo geral é avaliar a viabilidade e as limitações dessa integração. Os objetivos específicos são: (i) implementar um processo rastreável do entendimento aos resultados; (ii) avaliar a validade das decisões, dos fluxos e das asserções; (iii) distinguir falhas da aplicação, do ambiente e da própria pipeline; e (iv) incorporar supervisão e reutilização de feedback humano sem tratá-lo como prova de ocorrências futuras.

A contribuição é um instrumento executável e uma avaliação formativa baseada em seus artefatos. Na rodada central, Dopa, Expense Tracker e Typing Game produziram seis testes, dos quais cinco passaram. A análise qualitativa encontrou um candidato forte a falha de execução no Dopa e um falso positivo no Typing Game. Também identificou testes aprovados que não verificavam o objetivo anunciado. Uma revisão posterior separou descoberta da interface e análise de defeitos; no Form Validator, dois fluxos foram admitidos após corrigir uma rejeição excessiva. Esses resultados delimitam o valor atual da abordagem e as condições necessárias para aprimorá-la.

O restante do artigo apresenta a fundamentação e os trabalhos relacionados na Seção 2, a metodologia na Seção 3, a implementação na Seção 4, os resultados na Seção 5, as ameaças à validade na Seção 6 e as conclusões na Seção 7.

2. Referencial Teórico e Trabalhos Relacionados

2.1. Da execução de interações à avaliação de comportamento

Um caso de teste associa condições iniciais, ações e resultados esperados. Um oráculo é o critério utilizado para decidir se o resultado observado é aceitável. No E2P, critérios em linguagem natural antecedem a automação e podem ser revisados. Essa separação é necessária porque a validade sintática do script, a existência do seletor e a aprovação da execução não garantem correspondência com a intenção do caso.

A revisão de Wang et al. [Wang et al. 2024] organiza aplicações de LLMs em diferentes atividades de teste e destaca desafios de sua integração com outras técnicas. A distinção entre gerar entradas, produzir asserções e avaliar resultados orienta este estudo: uma melhoria em uma dessas atividades não é automaticamente transferível às demais. Em particular, medir somente quantos testes passam favorece testes permissivos, que podem permanecer verdes mesmo diante de um defeito.

Na avaliação de hipóteses, distinguem-se três afirmações: uma interação ocorreu; seu efeito foi reproduzido; esse efeito viola uma expectativa justificável. As duas primeiras são observacionais. A terceira requer evidência normativa, como um requisito, uma

mensagem explícita da interface ou uma regra de domínio sustentada pelo projeto. O código auxilia na explicação causal, mas não constitui sozinho uma especificação infalível: ele também pode conter o defeito investigado. Essa distinção fundamenta a análise dos falsos positivos apresentada adiante.

2.2. Agentes de interface e testes baseados em uso

WebArena oferece aplicações e tarefas web para avaliar conclusão funcional, em vez de apenas a plausibilidade do texto produzido. No experimento publicado, o melhor agente GPT-4 obteve 14,41% de sucesso, frente a 78,24% dos participantes humanos [Zhou et al. 2024]. Esses números pertencem àquele protocolo e não são uma referência quantitativa direta para o E2P. WorkArena e BrowserGym enfatizam ações padronizadas, observações multimodais e tarefas de software corporativo [Drouin et al. 2024]. Ambos reforçam a necessidade de ambientes registráveis e medidas ligadas ao comportamento efetivamente realizado.

No domínio móvel, GPTDroid estrutura a exploração como interação iterativa entre modelo e aplicação, utilizando informações da interface, feedback e memória relacionada às funcionalidades [Liu et al. 2024]. O E2P adota o princípio de observar novamente após agir, mas restringe decisões a identificadores de ações catalogadas e concentra-se em aplicações web. A memória de feedback humano entre execuções, descrita neste artigo, é diferente da memória da trajetória usada para escolher a próxima ação.

Avigust transforma vídeos de uso em uma representação de estados para apoiar a geração de testes em aplicações móveis [Zhao et al. 2022]. A aproximação com o E2P está na valorização de jornadas observadas. A diferença é a origem dessas jornadas: no E2P, elas são produzidas pela exploração guiada pelo modelo, e não fornecidas previamente como vídeos de usuários. Isso reduz a necessidade de uma gravação inicial, mas transfere ao explorador a responsabilidade de alcançar comportamentos úteis.

2.3. Geração, feedback e verificação de requisitos

LIBRO gera testes de reprodução a partir de relatos de defeitos e utiliza pós-processamento para avaliar e ordenar candidatos [Kang et al. 2023]. Enquanto esse trabalho parte de um problema relatado, o E2P também propõe hipóteses a partir da interface explorada. Portanto, além da reprodução, precisa avaliar se a própria alegação de erro é válida.

Ferreira et al. [Ferreira et al. 2025] apresentam um estudo industrial com geração de cenários Gherkin a partir de histórias de usuário e conversão para testes Cypress usando HTML. Os cenários foram considerados úteis em 95% das avaliações; 60% dos testes puderam ser utilizados como gerados, enquanto os demais exigiram ajustes, nova geração ou descarte. Esse resultado apoia o uso de artefatos intermediários revisáveis. Diferentemente daquele estudo, o E2P precisa inferir oportunidades de teste de um projeto e de sua execução, sem pressupor um conjunto completo de histórias de usuário.

Panta combina análise estática de fluxo e cobertura dinâmica para orientar iterativamente a geração de testes unitários [Gu et al. 2026]. TestWeaver usa recortes de programa, testes relacionados e anotações de execução para fornecer contexto dirigido ao modelo [Le et al. 2026]. Essas abordagens motivam o uso de sinais verificáveis e de con-

texto selecionado. Contudo, a cobertura de objetivos de interface do E2P não é cobertura de linhas ou ramos e não deve ser comparada numericamente a essas medidas.

GUISpector verifica requisitos em linguagem natural sobre protótipos de interface com um agente multimodal e disponibiliza feedback para supervisão [Kolthoff et al. 2025]. O E2P compartilha o interesse na rastreabilidade entre requisito e trajetória, mas também investiga a descoberta dos próprios casos. Isso torna a procedência da expectativa um problema central: inferir um requisito exige cautela adicional em relação a verificar um requisito fornecido.

SpecOps decompõe o teste de agentes GUI em geração, preparação, execução e validação. A avaliação relatada encontrou 164 defeitos em cinco agentes, com F1 de 0,89 [Ahmed et al. 2026]. Seu objeto são agentes de IA em ambientes reais, enquanto o E2P avalia aplicações web. A principal relação metodológica é a separação dos estágios; nem os resultados nem a autonomia daquela arquitetura podem ser atribuídos ao protótipo aqui estudado.

2.4. Posicionamento do trabalho

O E2P integra entendimento do repositório, descoberta da interface, casos revisáveis, compilação de jornadas e análise de hipóteses em uma ferramenta local. Não apresenta uma nova técnica de treinamento nem demonstra superioridade sobre os trabalhos citados. Sua contribuição específica está em tornar observáveis as perdas de informação entre estágios, permitindo analisar por que uma jornada plausível pode resultar em uma asserção fraca ou em um falso relato de defeito. A revisão é narrativa e direcionada a essas decisões arquiteturais; não constitui revisão sistemática da literatura.

3. Metodologia

3.1. Delineamento e questões de pesquisa

A pesquisa é aplicada e exploratória, com desenvolvimento iterativo e avaliação formativa de casos. As execuções são instrumentadas, mas não constituem um experimento controlado de comparação entre modelos: configuração e implementação foram alteradas ao longo do desenvolvimento. A unidade de análise é uma execução, identificada por aplicação, instante de criação, configuração do modelo, modo de acesso, trajetória e artefatos. O recorte documental considera a implementação e os registros disponíveis em 13 de setembro de 2026.

As questões de pesquisa são: **QP1**, quais etapas produzem decisões e artefatos executáveis fundamentados na aplicação; **QP2**, até que ponto testes aprovados e hipóteses reproduzidas correspondem a verificações funcionais válidas; **QP3**, quais limitações de segurança, observação e inferência restringem a exploração; e **QP4**, que evidência sustenta os aprimoramentos introduzidos e o que ainda exige avaliação adicional. Essas questões permitem analisar o instrumento sem atribuir causalidade a uma simples troca de modelo.

3.2. Processo e unidade de evidência

A Figura 1 sintetiza o processo implementado. Documentação, manifestos e trechos selecionados de código alimentam a inspeção. O ambiente é iniciado ou conectado a uma

URL existente. O explorador observa a interface e escolhe ações permitidas. Após essa descoberta, hipóteses são propostas, criticadas e submetidas à reprodução; a trajetória também fundamenta os fluxos que serão revisados e convertidos em testes.

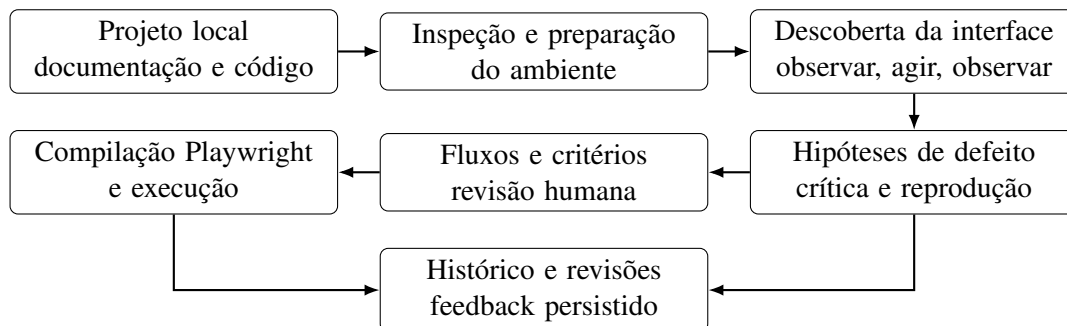


Figura 1. Processo do E2P; o histórico pode contextualizar análises futuras.

A ordem efetiva merece destaque: a análise de defeitos ocorre ao final da exploração, antes do planejamento dos fluxos, e seus resultados reaparecem nos relatórios. Não há atualmente um agente adversarial irrestrito que, após a suíte, explore novas jornadas para caçar bugs. O componente implementado analisa evidências coletadas e reproduz ações conhecidas. A descoberta inicial deve compreender usos ordinários; uma mensagem de validação encontrada nesse percurso não é, por si só, um defeito.

Cada ação concluída é associada a seu identificador, operação, justificativa, observação anterior e posterior, rota e indícios de mudança. A expectativa prevista pelo explorador é uma hipótese sobre a navegação, não um oráculo autoritativo. Controles bloqueados, decisões inválidas, erros de execução e motivo de término são mantidos como limitações da trajetória. Asserções e hipóteses devem ser avaliadas em relação a essa evidência, não apenas ao título gerado pelo modelo.

3.3. Seleção dos casos e configuração

A rodada central reúne Dopa, uma aplicação de catálogo, e os exemplos Expense Tracker e Typing Game do repositório Vanilla Web Projects [pieceofhell 2026, Traversy 2026]. A seleção foi intencional: Dopa expõe inicialização e interface dinâmica; Expense Tracker envolve entrada numérica, saldo e armazenamento local; Typing Game envolve seleção, pontuação e tempo. Form Validator, do mesmo repositório de exemplos, foi usado posteriormente para investigar uma falha específica de admissão de fluxos. Os casos não foram sorteados e não representam toda a população de aplicações web moderadas.

A inferência recente utiliza uma quantização local de 27 bilhões de parâmetros, identificada no ambiente como Unsloth IQ1_M e servida por llama.cpp [ggml-org 2026]. O arquivo de pesos é Qwen3.8-27B-UD-IQ1_M.gguuf; o alias experimental configurado é qwen3.8-v1-27b-iq1m-64k. O ambiente registra Windows, processador Ryzen 7 7800X3D, Radeon RX 9070 XT com 16 GB e 32 GB de RAM. O inicializador usa llama.cpp b10819 com Vulkan, contexto de 65.536 tokens, um slot, camadas na GPU, Flash Attention, cache KV Q8 e projetor visual BF16.

A janela configurada não significa que todo pedido utilize 65.536 tokens, nem assegura compreensão de todo o conteúdo. O adaptador solicita saída estruturada e limita a resposta conforme a etapa. A avaliação central foi operada pela interface gráfica

do E2P no navegador integrado disponível; internamente, a exploração usa Playwright com Chromium sem janela, em contexto isolado. Não se caracteriza, portanto, como um experimento com Firefox nem como estudo de usabilidade com usuários finais.

3.4. Procedimento e medidas

O procedimento consistiu em selecionar o projeto, inspecioná-lo, revisar o ambiente inferido, explorar sua interface, solicitar fluxos, aprovar casos, gerar testes e executá-los. Os registros foram examinados progressivamente, confrontando critérios, passos, asserções e diagnósticos. No Typing Game, por exemplo, três dos cinco fluxos propostos foram selecionados na revisão operada pela interface. Trata-se de avaliação assistida por ferramentas e IA, sem painel independente de avaliadores ou medição de concordância entre especialistas.

As medidas quantitativas incluem ações concluídas, estados distintos segundo o identificador do explorador, término, fluxos admitidos, testes aprovados e falhos, hipóteses retidas e tempo de inferência isolado do relógio da aplicação. A pontuação de cobertura QA é uma heurística ponderada de objetivos classificados como cobertos, parciais ou não cobertos. Ela é analisada como saída do instrumento, não como percentual de funcionalidades corretas ou de código coberto.

Na análise qualitativa, uma hipótese é classificada como candidato sustentado, comportamento esperado/falso positivo, problema ambiental ou inconclusiva. Um erro reproduzido pode ser um candidato forte, mas sua confirmação como defeito do produto requer justificar o comportamento esperado e descartar uma configuração incompatível. Não são calculados precisão, revocação ou F1 do E2P: faltam um conjunto de defeitos de referência e uma rotulagem independente e exaustiva. A ausência de hipótese retida não demonstra ausência de defeitos.

3.5. Rastreabilidade e reprodução

Os identificadores completos e a correspondência entre afirmações, relatórios e arquivos foram preservados no material de acompanhamento desta revisão. As execuções centrais de 8 de setembro de 2026 estão em diretórios iniciados por `dopa-`, `expense-tracker-` e `typing-game-`, com horários UTC de criação 11:42:34, 11:53:53 e 11:59:57. O reteste bem-sucedido de planejamento do Form Validator foi criado em 13 de setembro, às 19:12:00 UTC. Esses registros não foram substituídos por novas execuções durante a redação.

Para repetir o procedimento, é necessário preservar o projeto-alvo e suas dependências, iniciar o modelo pelo script fornecido, executar o E2P, selecionar o mesmo modo de acesso e repetir as etapas de revisão. Os arquivos `exploration.json`, `approved-flows.json`, os scripts gerados e os resultados Playwright permitem auditar o percurso. Entretanto, a versão nominal do protótipo não distingue todas as alterações locais. A ausência de uma revisão imutável comum a todas as rodadas e a evolução das dependências limitam a reprodução exata; um pacote experimental congelado permanece necessário.

4. Descrição da Implementação

4.1. Serviços, ambiente e integração com modelos

O E2P utiliza Node.js, Express, interface em HTML/CSS/JavaScript, Cheerio para inspeção estática e Playwright para observação e execução [Microsoft 2026]. Serviços separados tratam inspeção, ambiente, provedor, exploração, cobertura, hipóteses, planejamento, geração e histórico. A aplicação local é disponibilizada, por padrão, na porta 4318. Cada execução mantém artefatos em um diretório próprio, evitando inserir testes gerados no código-fonte do alvo. Isso não torna o ambiente imutável: a instalação de dependências e a inicialização do alvo podem produzir arquivos e efeitos próprios.

O ambiente pode ser servido como conteúdo estático, iniciado por comando, acessado por URL externa já disponível ou submetido apenas à análise. O comando e a URL inferidos são revisáveis. O adaptador preserva compatibilidade com diferentes provedores, mas o perfil recente usa diretamente llama.cpp, sem Ollama como intermediário. No fluxo convidado, imagens podem complementar o DOM quando o provedor local suporta visão. A presença de captura para o usuário e o envio de imagem ao modelo são mecanismos distintos, explicitados na configuração e nos registros.

4.2. Descoberta da interface e orçamento

O explorador oferece ações como clicar, preencher, selecionar e pressionar uma tecla sobre controles conhecidos. O modelo escolhe um identificador e parâmetros limitados; não fornece seletores arbitrários para execução irrestrita. Decisões e respostas JSON inválidas entram em um ciclo limitado de correção. A política AI-first exige participação válida do modelo nas etapas semânticas e interrompe a progressão quando o contrato não é satisfeito, preservando a evidência parcial.

Um plano de objetivos QA é derivado da inspeção e da observação inicial. Ele inclui, quando há evidência, entrada e confirmação, seleção, navegação, validação, temporização, persistência e estados calculados. Foram adicionadas ações sintéticas limitadas de espera e recarga. A espera temporal é limitada a 12 segundos; a recarga depende de indícios de persistência e de uma mudança prévia de estado. Um orçamento adaptativo limita a exploração, com tetos de 20 ações e 180 segundos na configuração dos registros analisados.

A versão usada em 8 de setembro enfatizava fronteiras e entradas mínimas. A revisão de 13 de setembro passou a priorizar jornadas ordinárias com valores fictícios plausíveis: os objetivos QA fornecem contexto, sem obrigar o explorador a provocar erros. Uma entrada já utilizada pode ser corrigida uma vez após feedback de validação associado ao próprio campo, seguida de nova confirmação. Ações proibidas continuam fora do catálogo. Isso melhora a separação de responsabilidades sem remover a coleta de exceções e erros visíveis durante a descoberta.

Para reduzir a interferência da latência do modelo, o relógio da página convidada é pausado durante a inferência e retomado antes da ação. Esperas explícitas avançam normalmente. Esse mecanismo não congela serviços externos nem elimina o custo total da execução. O orçamento global ainda pode ser consumido pela inferência, mesmo quando o tempo da aplicação está isolado, como observado no Dopa. A disponibilidade da instrumentação e o tempo isolado são registrados.

4.3. Planejamento e geração de testes

Os fluxos e critérios propostos devem remeter a controles e transições registrados. Propostas não fundamentadas são rejeitadas; solicitações menores por transição podem recuperar parte do planejamento sem inventar evidência. A revisão permite selecionar e editar critérios antes de gerar a suíte. A correção recente distingue observar mensagens relacionadas a senha de propor preenchimento de credenciais: o primeiro pode ser legítimo em um formulário parcialmente explorado; o segundo permanece sujeito à política de acesso.

No caminho convidado atual, o gerador compila diretamente as jornadas executadas em Playwright, registrando o modo `model-journey-compiled`. Não se trata de código livre integralmente escrito pelo LLM, nem apenas de uma alternativa acionada quando tal código falha. O modelo participa da escolha da trajetória e da autoria semântica dos casos; o compilador determina a estrutura executável e as verificações disponíveis. Essa distinção de proveniência é essencial para não atribuir ao modelo resultados produzidos por regras determinísticas.

O contrato de seletores impede utilizar controles não observados, mas não resolve toda a validade do teste. Uma asserção pode consultar um campo verdadeiro e ainda não verificar o objetivo do caso. O executor registra falhas, anexos e evidências; a interpretação final recebe as lacunas QA, mas continua sujeita a exageros semânticos. A avaliação verifica, portanto, também o código gerado, em vez de considerar a narrativa final como resultado experimental autoritativo.

4.4. Hipóteses, crítica e reprodução

A análise de defeitos recebe estados, ações, evidências visuais quando permitidas e diagnósticos de execução. Um papel autor propõe candidatos e um papel crítico avalia se a ação ocorreu, se a expectativa tem fundamento e se o efeito sustenta a alegação. Regras determinísticas filtram candidatos e podem originar hipóteses baseadas em erros de execução. A proveniência diferencia candidatos do modelo e dos diagnósticos; dois papéis não significam necessariamente dois modelos diferentes ou julgamentos estatisticamente independentes.

As hipóteses elegíveis passam por reprodução em uma nova sessão do navegador, repetindo ações registradas e comparando o efeito. Isso reduz dependência de estados residuais e fornece evidência de repetibilidade. Porém, não prova que a expectativa do autor era correta, nem corrige automaticamente uma justificativa incorreta aceita pelo crítico. A interface preserva o caráter de hipótese até revisão, e o artigo não equipara confiança atribuída pelo modelo a confirmação do defeito.

4.5. Histórico, feedback humano e evidências

O histórico reúne execuções antigas e recentes, inclusive incompletas, sem iniciar novamente o alvo ou substituir o projeto ativo. Resultados de execução são persistidos antes do enriquecimento textual pelo modelo, evitando perder testes concluídos quando a interpretação falha. Capturas, vídeos e rastros Playwright são agrupados por teste; anexos indisponíveis apresentam estado explícito e tentativa de recarga. A interface também recebeu ajustes de largura, espaçamento e cores. Essas melhorias apoiam inspeção, mas não foram avaliadas em estudo formal de usabilidade.

Hipóteses podem receber revisão humana com justificativa: confirmação, comportamento esperado/falso positivo, problema ambiental ou inconclusiva. As decisões são eventos datados; corrigir uma decisão acrescenta um novo evento sem apagar os anteriores. Execuções futuras do mesmo caminho absoluto normalizado recuperam até 20 revisões distintas mais recentes para contextualizar autor e crítico. O histórico é tratado como informação não autoritativa, e não como instrução ou evidência da execução corrente.

Esse recurso é recuperação de contexto, não treinamento do modelo. Mover o projeto para outro caminho cria uma identidade diferente, e reformulações do título ou da expectativa podem não ser unificadas. Os testes do recurso demonstram persistência e entrega de contexto, mas ainda não demonstram redução de falsos positivos em novas execuções. A avaliação de eficácia precisa comparar análises com e sem feedback, mantendo o restante das condições fixo.

4.6. Segurança e testes do instrumento

O catálogo e as políticas de rede restringem operações potencialmente mutáveis, credenciais e navegação inadequada. O modo autenticado é uma prova de conceito somente para leitura, com referências a segredos por variáveis de ambiente, sessões isoladas e quarentena de artefatos. Vídeos e rastros são desabilitados nesse modo, e a análise visual de defeitos apresentada neste estudo está indisponível em sessões autenticadas. Essas medidas reduzem risco, mas não constituem uma garantia absoluta de confidencialidade ou ausência de efeitos.

Há um custo funcional importante: a política convidada ainda bloqueia entradas necessárias a alguns exemplos, como o campo numérico de valor no Expense Tracker e campos de e-mail e senha no Form Validator. Um encerramento por esgotamento do catálogo significa esgotamento das ações permitidas, não compreensão completa do produto. A documentação mais recente registra 96 testes automatizados do próprio E2P aprovados, envolvendo protocolo, compilação, diagnósticos, histórico e integração com navegador. Esses testes são distintos das suítes geradas para as aplicações da avaliação.

5. Resultados e Discussão

5.1. Rodada central em três aplicações

A Tabela 1 reúne as execuções de 8 de setembro. Os estados são os identificadores distintos produzidos pelo explorador, não telas semanticamente independentes validadas por pessoas. Ações indisponíveis e observações intermediárias podem elevar a quantidade de estados sem gerar uma jornada funcional nova.

Tabela 1. Execuções com o modelo IQ1_M; testes aprovados sobre executados.

Aplicação	Ações	Estados	Testes	Hipóteses	Término
Dopa	2	6	0/1	1	limite de tempo
Expense Tracker	4	2	2/2	0	catálogo esgotado
Typing Game	6	7	3/3	1	catálogo esgotado
Total	12	15	5/6	2	—

No **Dopa**, a exploração preencheu o campo de busca e pressionou Enter. Dez tentativas de clique ficaram indisponíveis entre observação e execução, compatíveis com a substituição de controles durante a atualização da interface. O fluxo de busca gerou um teste que falhou pela presença de uma sobreposição de erro do Vite. A hipótese retida descreveu uma exceção na otimização de imagens: `Cannot read properties of undefined (reading 'fetch')`, associada a `worker/index.ts:35`, resposta HTTP 500 e reprodução da assinatura em sessão limpa.

Esse conjunto sustenta um candidato forte a falha de execução ou configuração no ambiente utilizado. Não demonstra um defeito funcional da busca, nem permite afirmar confirmação independente no ambiente de referência do projeto. Três outros candidatos foram rejeitados. O diagnóstico é útil porque aponta um impedimento visível antes de interpretar o resultado de uma jornada como comportamento normal do produto.

No **Expense Tracker**, ocorreram preenchimento de texto, Enter, ativação de adição e recarga. O campo *Amount*, numérico, ficou bloqueado pela política. Consequentemente, nenhuma transação válida foi criada e o saldo permaneceu em zero. Os dois testes passaram, mas a recarga verificou essencialmente o estado inicial e a perda do texto não submetido. Isso não comprova persistência de transações. Nenhuma hipótese foi retida, e um candidato foi rejeitado: o resultado é ausência de achado nessa exploração limitada, não evidência de correção da função financeira.

No **Typing Game**, o modelo preencheu parte da palavra, pressionou Enter, selecionou dificuldade, abriu configurações, esperou sete segundos e recarregou a página. O estado de tempo esgotado apareceu após a espera explícita. Dos cinco fluxos propostos, foram selecionados os relativos a tempo, dificuldade e configurações. Foram descartados uma entrada parcial pouco representativa e um fluxo de recarga que exigia indevidamente uma palavra aleatória diferente.

Os três testes selecionados passaram, mas suas asserções finais verificaram o mesmo valor residual do campo de digitação, em vez de verificar diretamente o estado de fim de jogo, a dificuldade e a abertura do painel. Logo, três aprovações não equivalem a três objetivos funcionais adequadamente testados. A falha está na passagem de intenção para asserção, ainda que a trajetória contenha estados úteis.

A hipótese retida no jogo alegava que o cronômetro deveria parar enquanto a digitação estivesse incompleta. O comportamento foi reproduzido, mas é esperado em um jogo contra o tempo e é coerente com o código e a descrição do exemplo. Trata-se de falso positivo na análise dos registros. Autor e crítico aceitaram a mesma premissa incorreta; a reprodução apenas confirmou que o relógio continuava. Na rodada central, portanto, há um candidato forte, um falso positivo e nenhum defeito confirmado por revisão independente.

5.2. Cobertura heurística e isolamento temporal

A Tabela 2 explicita o que o instrumento creditou e o que a análise dos artefatos permite concluir. Não se deve interpretar esses percentuais como cobertura funcional medida. No Dopa, números alterados após busca receberam crédito de estado calculado, sem demonstrar uma operação de cálculo. No jogo, uma contagem regressiva também não equivale ao cálculo correto de pontuação. No Expense Tracker, executar recarga não comprova preservar uma transação que nunca foi criada.

Tabela 2. Objetivos QA creditados pelo E2P e ressalvas da análise.

Aplicação	Cobertos	Índice	Limitação observada
Dopa	3/5	68,0%	Mudança numérica não demonstra cálculo; ativação principal não foi coberta.
Expense Tracker	4/7	76,4%	Recarga ocorreu sem transação válida; persistência de domínio não foi verificada.
Typing Game	7/9	88,3%	Timer foi observado, mas asserções não verificaram os objetivos dos três casos.

O isolamento temporal registrou 22 pausas, totalizando 148.094 milissegundos de inferência: 87.653 no Dopa, 22.847 no Expense Tracker e 37.594 no Typing Game. Há evidência operacional de que o relógio instrumentado não avançou silenciosamente durante essas decisões. Não há comparação controlada suficiente para quantificar seu efeito sobre a detecção de defeitos. No Dopa, o custo de inferência continuou consumindo o orçamento total, limitando a quantidade de ações concluídas.

5.3. Refinamento da descoberta no Form Validator

Uma execução de 13 de setembro preencheu *Username* com um único caractere e pressionou Enter. A interface apresentou mensagens de validação, e o catálogo esgotou as ações permitidas. O planejamento foi interrompido por rejeição dos fluxos. A investigação encontrou dois problemas do instrumento: a descoberta incentivava entradas de fronteira antes de compreender uma jornada comum, e o filtro de fundamentação rejeitava menções a senha mesmo quando apenas descreviam mensagens observadas em outros campos.

A revisão passou a usar valores fictícios ordinários, admitiu uma correção limitada após validação e distinguiu observação de erro de proposta de entrada de credenciais. A primeira tentativa de reteste revelou também que JSON malformato escapava do ciclo de correção; esse tratamento foi ajustado. Na segunda tentativa, *Username* recebeu *sampleuser*, seguido de Enter, com duas ações, três estados e nenhuma decisão inválida. Dois fluxos foram admitidos: *Registration Form Validation Journey* e *Username Field Fill Observation*. Dois candidatos de defeito foram rejeitados, e nenhum foi retido.

Esse resultado demonstra recuperação operacional de exploração e planejamento nesse caso. Os dois fluxos não foram compilados e executados em uma nova suíte nessa avaliação. Tampouco houve registro completo de usuário: e-mail, senha e submissão direta continuaram limitados pela política. A comparação é um diagnóstico antes/depois, não uma estimativa estatística de melhoria. O Dopa não foi reexecutado após esse último ajuste; a preservação do tratamento de erros de execução é apoiada por testes de regressão, não por um novo benchmark daquele alvo.

5.4. Modelo local e comparação histórica

O perfil de inferência foi selecionado em medições anteriores. O relatório local de 5 de setembro registra aproximadamente 51,9 tokens/s de geração curta para IQ1_M e 43,1 tokens/s para IQ2_S, ambos com Vulkan. Em um pedido com 51.560 tokens de entrada,

a recuperação de uma chave foi correta, mas a geração com cache Q8 caiu para 41,4 tokens/s. Portanto, suportar mais de 50 mil tokens de contexto e alcançar 50 tokens/s em uma chamada curta não significa atingir ambas as condições simultaneamente com o contexto ocupado.

Essas medições são de desempenho do ambiente, não uma avaliação abrangente da compreensão em contexto longo ou da qualidade de QA. O registro histórico de comparação com modelos menores também alterou esquema JSON, limites de saída e agrupamento de ações. Ele orientou a engenharia, mas não isola o efeito do modelo ou da quantização. Por essa razão, os antigos resultados com `qwen3:8b` em Janvas e TodoMVC não são combinados com a rodada atual em uma taxa global de sucesso ou em uma alegação de superioridade.

5.5. Respostas às questões e prioridades

Quanto à **QP1**, a pipeline demonstrou integração operacional: ações observadas puderam fundamentar casos e gerar suítes executáveis. O reteste do formulário mostrou que uma política excessiva do próprio instrumento pode impedir esse caminho, mesmo com evidência útil disponível. Quanto à **QP2**, a aprovação de cinco testes não assegurou correspondência com suas intenções, e a reprodução de uma hipótese não impediu o falso positivo do jogo.

Para a **QP3**, destacam-se bloqueio de entradas centrais, instabilidade de controles e consumo do orçamento pela inferência. Quanto à **QP4**, há evidência direta de funcionamento de espera, recarga, isolamento temporal e recuperação do planejamento. Histórico e feedback persistido estão implementados e cobertos por testes, mas sua eficácia contra falsos positivos permanece não medida. As alterações de interface não sustentam, isoladamente, conclusões sobre precisão ou produtividade.

As prioridades decorrentes dos casos são: vincular cada critério a uma asserção que verifique seu efeito; medir cobertura por mudança de estado relevante, e não pela mera presença de ação; permitir dados sintéticos tipados sob limites seguros; tornar o crítico capaz de procurar explicações de comportamento normal; e separar custo de inferência, tempo do alvo e tempo total. Estabilização e nova catalogação de controles também são relevantes para interfaces dinâmicas. Essas propostas são trabalhos futuros, não capacidades já demonstradas pela versão atual.

6. Ameaças à Validade

A validade de construção é limitada pelas métricas substitutas: estados distintos, objetivos heurísticos e testes aprovados podem superestimar a profundidade da avaliação. A classificação de defeitos depende da procedência da expectativa e não apenas da repetição do efeito. Não houve conjunto de referência que permitisse medir defeitos perdidos, nem avaliação independente com concordância entre revisores. A revisão assistida por IA dos artefatos está sujeita a vieses e requer ratificação dos responsáveis pelo estudo.

A validade interna é afetada pela evolução simultânea de prompts, políticas, compilador e modelo. Rodadas em datas diferentes não permitem atribuir melhorias exclusivamente à quantização ou ao refinamento mais recente. Temperatura baixa e saídas estruturadas não eliminam a variação. O contexto limpo reduz resíduos no navegador, mas não restaura necessariamente serviços externos. Pausar o relógio altera as condições

temporais de uso e não equivale a um usuário humano real ou a controle completo do ambiente.

A validade externa é limitada pela amostra intencional, pelos exemplos didáticos, pelo modo convidado e pelo uso interno de Chromium. A avaliação não demonstra generalização para aplicações moderadas autenticadas, integrações complexas ou outros navegadores. Uma política de somente leitura pode deixar funcionalidades centrais fora da amostra e não garante ausência de efeitos em aplicações que alterem dados por requisições de leitura.

A reprodutibilidade depende de preservar pesos, projetor, configuração, versões do alvo e artefatos. O nome do modelo e o tamanho da janela não substituem identificadores imutáveis, e a versão nominal do E2P não representa todas as revisões locais. Além disso, recuperar avaliações anteriores pode ajudar ou propagar um julgamento incorreto. Esse recurso deve ser avaliado com separação entre dados usados no feedback e novos casos, evitando apresentar memorização de decisões anteriores como generalização.

7. Conclusões e Trabalhos Futuros

O E2P materializa uma pipeline local de QA que integra entendimento de projeto, descoberta da interface, critérios revisáveis, compilação de jornadas Playwright, execução e análise de hipóteses. A implementação atual inclui visão opcional, inferência local quantizada, evidências por teste, diagnóstico e reprodução, histórico e feedback humano. Sua contribuição é tornar o processo auditável, permitindo distinguir uma decisão do modelo de uma transformação determinística ou de uma intervenção de revisão.

A avaliação em três aplicações produziu seis testes, cinco aprovados, mas identificou asserções semanticamente insuficientes. Na análise de defeitos, um erro do Dopa permaneceu como candidato forte de execução/configuração e uma alegação sobre o cronômetro do Typing Game foi classificada como falso positivo. O reteste do Form Validator demonstrou recuperação do planejamento após separar descoberta ordinária e análise de defeitos, sem comprovar cobertura completa do formulário.

Em relação à literatura, o trabalho reforça a utilidade de etapas intermediárias e feedback observável, mas não apresenta resultados quantitativamente comparáveis aos estudos controlados citados. A principal conclusão é que o E2P já serve como apoio à exploração e à organização de evidências, enquanto sua confiabilidade para QA autônomo permanece limitada pela transformação de observação em expectativa e de expectativa em asserção.

Como continuidade, propõe-se congelar versões do instrumento e dos alvos; definir critérios de referência revisados por pessoas; executar repetições pareadas com e sem cada melhoria; e testar versões defeituosas e corrigidas dos mesmos projetos. Devem ser avaliadas relevância dos casos, correspondência das asserções, sensibilidade a defeitos, falsos positivos, custo total e efeito do feedback histórico. O avanço esperado é melhorar essas medidas mantendo explícito o que foi explorado, o que foi realmente verificado e o que ainda depende de julgamento humano.

Declaração de uso de inteligência artificial

A ferramenta Codex, da OpenAI, foi utilizada para apoio à revisão e atualização deste artigo, incluindo organização textual, tradução do resumo, conferência bibliográfica, con-

fronto com código e artefatos experimentais e formatação LaTeX. A contribuição abrangeu todas as seções desta versão. Os registros de implementação e avaliação também incluem operações assistidas por IA; não são apresentados como estudo independente com participantes humanos. A responsabilidade pela exatidão, originalidade, interpretação dos resultados e aprovação da versão submetida permanece com o autor e seu orientador.

Referências

- Ahmed, S. Y., Feng, S., Bae, C., Barrus, C., and Zhang, X. (2026). SpecOps: A fully automated AI agent testing framework in real-world GUI environments. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. <https://doi.org/10.1145/3744916.3787778>.
- Drouin, A., Gasse, M., Caccia, M., Laradji, I. H., Del Verme, M., Marty, T., Boisvert, L., Thakkar, M., Cappart, Q., Vazquez, D., Chapados, N., and Lacoste, A. (2024). WorkArena: How capable are web agents at solving common knowledge work tasks? arXiv:2403.07718, versão 5. <https://arxiv.org/abs/2403.07718>.
- Ferreira, M., Viegas, L., Faria, J. P., and Lima, B. (2025). Acceptance test generation with large language models: An industrial case study. In *IEEE/ACM International Conference on Automation of Software Test (AST)*. <https://doi.org/10.1109/AST66626.2025.00007>.
- ggml-org (2026). *llama.cpp*. Software. <https://github.com/ggml-org/llama.cpp>. Acesso em: 13 set. 2026.
- Gu, S., Nashid, N., and Mesbah, A. (2026). LLM test generation via iterative hybrid program analysis. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. <https://doi.org/10.1145/3744916.3764553>.
- Kang, S., Yoon, J., and Yoo, S. (2023). Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. <https://arxiv.org/abs/2209.11515>.
- Kolthoff, K., Kretzer, F., Ponzetto, S. P., Maedche, A., and Bartelt, C. (2025). GUI-pector: An MLLM agent framework for automated verification of natural language requirements in GUI prototypes. arXiv:2510.04791. <https://arxiv.org/abs/2510.04791>.
- Le, C. C., Van, C. D., Vu, T. D., Vu, T. M. P., Phan, H. N., Phan, H. N., and Nguyen, T. N. (2026). TestWeaver: Execution-aware, feedback-driven regression testing generation with large language models. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. <https://arxiv.org/abs/2508.01255>.
- Liu, Z., Chen, C., Wang, J., Chen, M., Wu, B., Che, X., Wang, D., and Wang, Q. (2024). Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. <https://arxiv.org/abs/2310.15780>.
- Microsoft (2026). *Playwright: Assertions*. Documentação. <https://playwright.dev/docs/test-assertions>. Acesso em: 13 set. 2026.

- pieceofhell (2026). *Dopa*. Repositório de software. <https://github.com/pieceofhell/dopa>. Cópia local utilizada na avaliação de 8 set. 2026.
- Traversy, B. (2026). *Vanilla Web Projects*. Repositório de exemplos. <https://github.com/bradtraversy/vanillawebprojects>. Cópia local utilizada nas avaliações de set. 2026.
- Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., and Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*. Versão dos autores: <https://arxiv.org/abs/2307.07221>.
- Zhao, Y., Talebipour, S., Baral, K., Park, H., Yee, L., Khan, S. A., Brun, Y., Medvidović, N., and Moran, K. (2022). Avgust: Automating usage-based test generation from videos of app executions. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. <https://arxiv.org/abs/2209.02577>.
- Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2307.13854>.